magic_mz_x86 and magic_mz_x64

Background

We'll this is going to be a very short blog post. magic_mz_x86 and magic_mz_x64 are two malleable profile values one can set in since cobalt strike 2.4.3 . I haven't seen anyone talk about it and what are the possible values. I have searched internet to find anyone using different set of values. No one has ever published this. So here, I'll publish some details about it.

Why Change these values?

magic_mz_x86 and magic_mz_x64 malleable options are available in "Stage" block of Cobaltstrike malleable profile. They are responsible to change the MZ PE header in the shellcode you generate from CobaltStrike (staged or stageless). There is basic information provided on cobaltstrike blog post on how to change these values. One can change these values by providing a set of 2 (for x64) or 4(for x86) assembly instructions. The condition for the assembly instructions is that the resultant should be a no operation. For eg

```
inc eax
dec eax
```

Above instructions combined together result in a no operation

How to change values?

Default values as provided in the blog from cobalt strike are as follows

Option	Example	Description
allocator	HeapAlloc	Set how Beacon's Reflective Loader allocates memory for the agent. Options are: HeapAlloc, MapViewOfFile, and VirtualAlloc.
cleanup	false	Ask Beacon to attempt to free memory associated with the Reflective DLL package that initialized it.
magic_mz_x86	MZRE	Override the first bytes (MZ header included) of Beacon's Reflective DLL. Valid x86 instructions are required. Follow instructions that change CPU state with instructions that undo the change.
magic_mz_x64	MZAR	Same as magic_mz_x86; affects x64 DLL.
magic_pe	PE	Override the PE character marker used by Beacon's Reflective Loader with another value.
module_x64	xpsservices.dll	Same as module_x86; affects x64 loader

https://www.cobaltstrike.com/help-malleable-postex

To change these values here is a generic approach

For x86 - magic_mz_x86

For x86 we have to write 4 instructions (resulting to NOP) to fill up MZRE space. You can use any 4 x86 instructions which can fill 4 byte space and result in a resultant NO-OPERATION . This is how MZRE is created

x86-orig.asm

```
bits 32
section .text
global _start
 _start:
dec abp
pop edx
push edx
inc ebp
```

So if you'll compile the above asm, and do a hexdump of this you'll see MZRE.

```
┌──(kali㉿ kali)-[~]
└─$ cat x86-orig.asm
bits 32
section .text
global _start
_start:
 dec ebp
 pop edx
 push edx
 inc ebp

┌──(kali㉿ kali)-[~]
└─$ nasm x86-orig.asm

┌──(kali㉿ kali)-[~]
└─$ hexdump -c x86-orig
0000000   M   Z   R   E
0000004

┌──(kali㉿ kali)-[~]
└─$ hexdump -C x86-orig
00000000  4d 5a 52 45  |MZRE|
00000004

┌──(kali㉿ kali)-[~]
└─$
```

now to modify, change these 4 instructions to any instructions of 4 byte total length. For example

x86-modif.asm

```
bits 32
section .text
global _start
_start:
 dec eax
 inc eax
 dec ebx
 inc ebx
```

```
┌──(kali㉿kali)-[~]
└─$ cat x86-modif.asm
bits 32
section .text
global _start
_start:
 dec eax
 inc eax
 dec ebx
 inc ebx

┌──(kali㉿kali)-[~]
└─$ nasm x86-modif.asm

┌──(kali㉿kali)-[~]
└─$ hexdump -C x86-modif
00000000  48 40 4b 43                                    |H@KC|
00000004

┌──(kali㉿kali)-[~]
└─$
```

As mentioned above , now you can change magic_mz_x86 to "H@KC"

For x64 - magic_mz_x64

Similarly, for x64, now you need two instructions to fill up the 4 byte space. You can use something

x64-modif.asm

```
bits 64
section .text
global _start
_start:
 pop r9
 push r9
```

Compiling the same will result as following

```
┌──(kali㉿ kali)-[~]
└─$ cat x64-modif.asm
bits 64
section .text
global _start
_start:
 pop r9
 push r9

┌──(kali㉿ kali)-[~]
└─$ nasm x64-modif.asm

┌──(kali㉿ kali)-[~]
└─$ hexdump -c x64-modif
0000000   A   Y   A   Q
0000004

┌──(kali㉿ kali)-[~]
└─$ hexdump -C x64-modif
00000000  41 59 41 51                                    |AYAQ|
00000004

┌──(kali㉿ kali)-[~]
└─$
```

Now AYAQ can be used as a value in magic_mz_x64

What is the actual difference now

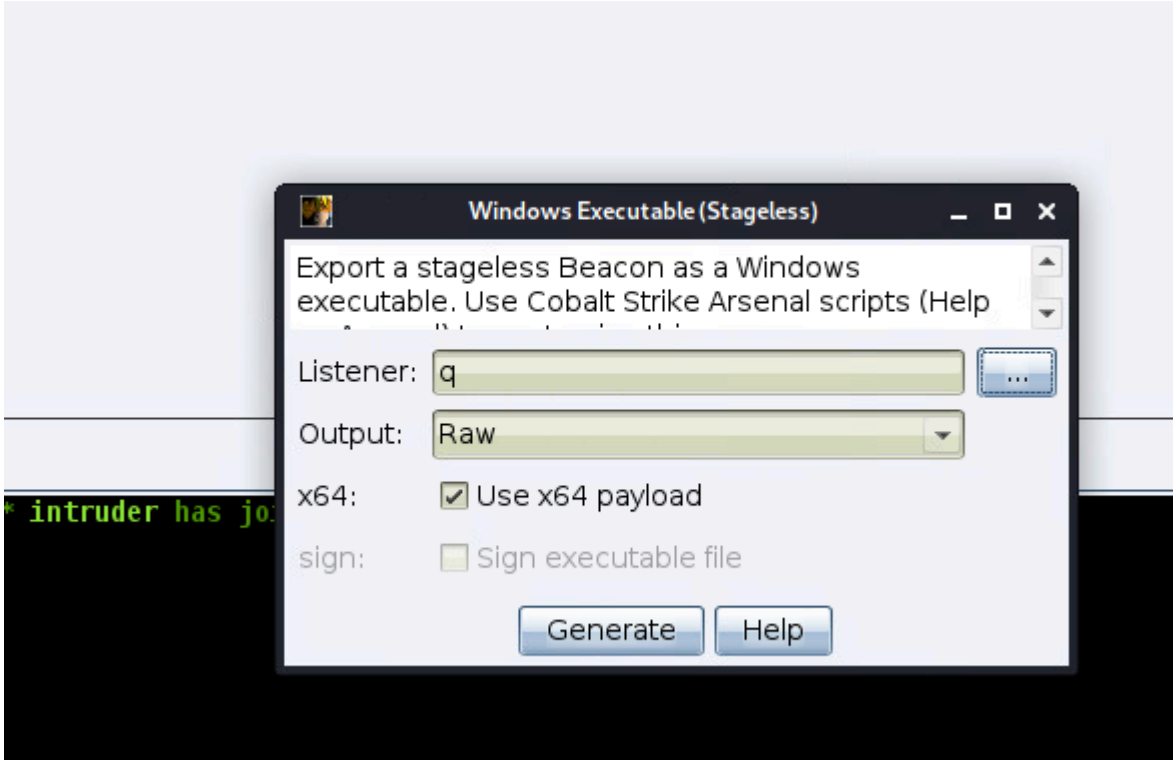
The difference is actually seen when you dump the stageless raw shellcode . You can see you MZ header change which helps evade EDR

With **Default** profile as below

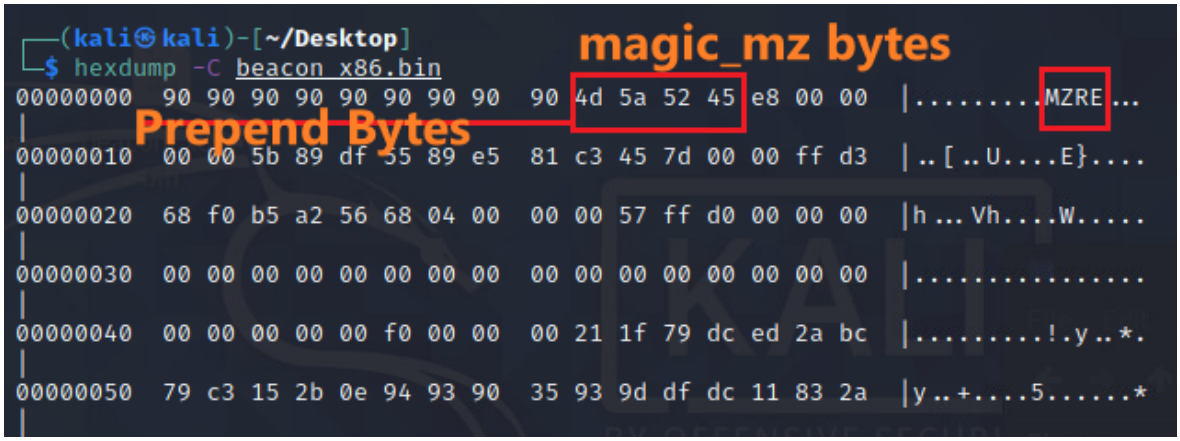
```
##      https://www.youtube.com/watch?v=AV4XjxYe4GM (
stage {

    # CS 4.2 added allocator and MZ header overrides
    set allocator      "VirtualAlloc"; # Options are:
    set magic_mz_x86   "MZRE";
    set magic_mz_x64   "MZAR";
    set magic_pe       "NO";
```

Profile with default magic_mz values

Following is the dump of stageless payload



Exporting Raw shellcode of unmodified profile



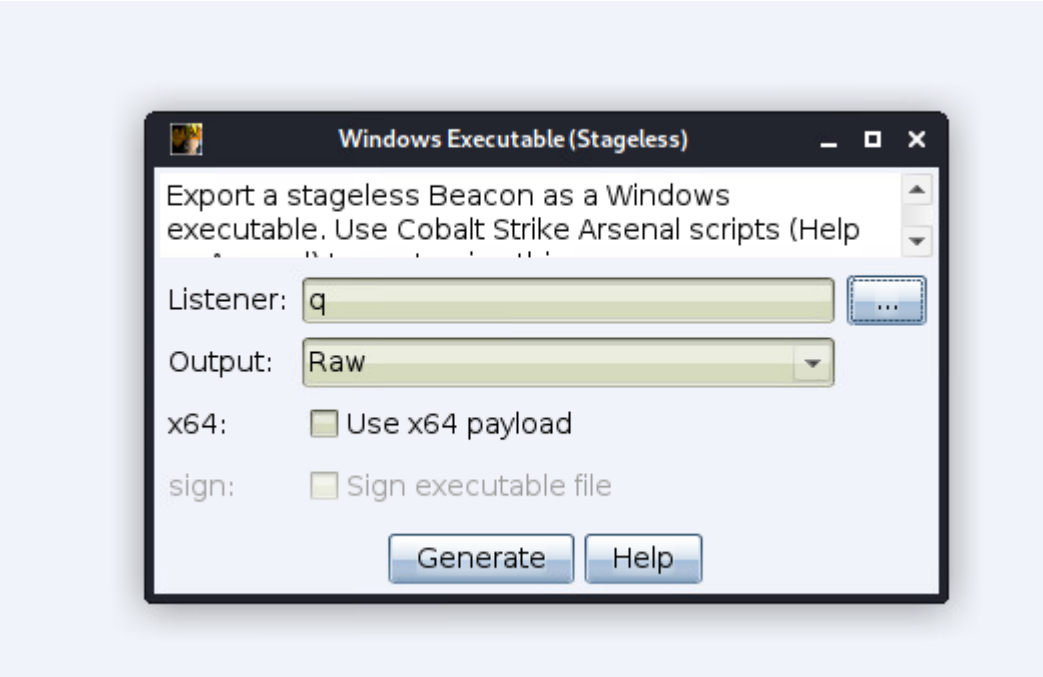
Hexdump of Deafult stageless x86 shellcode

Hexdump of Deafult stageless x64 shellcode

Now if we change profile values to custom values you can see the difference

```
stage {

    # CS 4.2 added allocator and MZ header overrides
    set allocator       "VirtualAlloc"; # Options are: HeapAlloc, MapViewOfFi
    set magic_mz_x86    "H@KC";
    set magic_mz_x64    "AYAQ";
    set magic_pe        "NO";
    set userwx          "false";
    set stomppe         "true";
```

Changing default values of magic_mz



Exporting Raw Shellcode after modifying the profile

```
┌──(kali㉿kali)-[~]
└─$ hexdump -C Desktop/beacon_x64_modif.bin
00000000  90 90 90 90 90 90 90 90  90 41 59 41 51 55 48 89  |........ AYAQUH.
|
00000010  e5 48 81 ec 20 00 00 00  48 8d 1d ea ff ff ff 48  |.H..  ...H......H
|
00000020  89 df 48 81 c3 cc 60 01  00 ff d3 41 b8 f0 b5 a2  |..H...`....A....
```

Hexdump of Modified stageless x64 shellcode

```
┌──(kali㉿kali)-[~]
└─$ hexdump -C Desktop/beacon_x86_modif.bin
00000000  90 90 90 90 90 90 90 90  90 48 40 4b 43 e8 00 00  |........H@KC..
|
00000010  00 00 5b 89 df 55 89 e5  81 c3 45 7d 00 00 ff d3  |..[..U....E}....
|
00000020  68 f0 b5 a2 56 68 04 00  00 00 57 ff d0 00 00 00  |h...Vh....W.....
```

Thanks

Thanks to @vysecurity for guidance and motivation

References

https://www.cobaltstrike.com/help-malleable-postex

Last updated 3 years ago

Was this helpful?